SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:50

Mode: In Class Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I DIVYASHREE *I* 192126004 *I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Scanned with OKEN Scanner

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^*\ abb$ c)$(a^*b^*)^*\ aba\ (a^*+b^*)^*$ d)$(a+ab)^*$ e) $(aba)^*+(a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d = “xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$:** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.

Scanned with OKEN Scanner

5 **Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:**

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

a) How many tokens are there in the given input string?

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Scanned with OKEN Scanner

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Scanned with OKEN Scanner

Analytical problem Solving.
CO1.

Name: Divyashree Koteeswaran
Reg.No: 192126004
Code: CSA1409-Slot-C.
Course: Compiler Design.

Q.1. Trace the processing of P = (a*b) + (b*c) * 2 thro' all phases of a compiler.

a) Tokens generated during lexical analysis.

→ The lexical analyzer divides the input into tokens

Lexeme	Token
P	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	Addition operator
(	Left parenthesis
b	Identifier.
*	multiplication operator.
c	Identifier.
)	Right parenthesis
*	Multiplication operator
2	Integer constant

89/100

03/04/26

∴, the complete statement contains

16 tokens

The RHS expression alone contains 14 tokens.

Scanned with OKEN Scanner

b) Construct the syntax tree during the parsing phase.

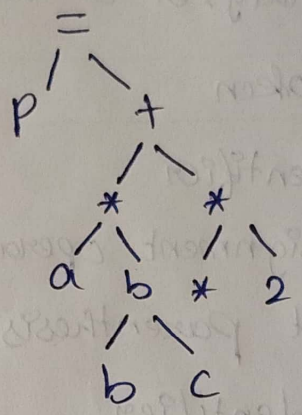
1st apply operator precedence.

$(a*b)+(b*c)*2$

multiplication has higher precedence than addition,

$(a*b)+((b*c)*2)$

Syntax tree is:



c) Generate the intermediate code for the given expression

⇒ Step 1.- $t_1 = a*b$

Step 2 - $t_2 = b*c$

Step 3 - $t_3 = t_2*2$

Step 4 - $t_4 = t_1 + t_3$

Step 5 - $p = t_4$.

final Intermediate code:

$t_1 = a*b$

$t_2 = b*c$

$t_3 = t_2*2$

$t_4 = t_1 + t_3$

$p = t_4$.

Scanned with OKEN Scanner

d) Apply suitable optimizations to the intermediate code.

⇒ $t_4 = t_1 + t_3$

$P = t_4$.

∴, $t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$P = t_1 + t_3$.

This reduces the number of instructions and temporary variables

Optimized intermediate code:

$t_1 = a * b$
$t_2 = b * c$
$t_3 = t_2 * 2$
$P = t_1 + t_3$;

e) Explain the significance of the final target code prod.

The final target code converts the optimized intermediate code into instructions that can be executed by the target machine.

Significance:

The target code:

1. Can be executed by the target processor.
2. Uses registers and machine instructions.
3. Performs the req. arithmetic operation.
4. Stores the final result in P.
5. Represents the final output of the compiler.

Scanned with OKEN Scanner

Q.2). for each of the following regular expressions, construct the language. write any five valid strings for each regular expression.

a) $(\varepsilon a a^*)^*$

W.K, $\varepsilon a^* a^* = a^* a^*$

The concatenation of two a^* expression is still a^*

∴, $L = \{a^n \mid n > 0\}$

five valid strings.

No.	String
1	ε
2	a
3	aa
4	aaa
5	aaaa

Answer: $\{\varepsilon, a, aa, aaa, aaaa, \ldots\}$.

b) $(a^* b)^* abb$

⇒ Step-1

$a^* b$ means:

- zero / more as
- followed by one b.

Step-2

$(a^* b)^*$

the language has the form:

$(a^* b)(a^* b) \ldots (a^* b)\, abb$

five valid strings.

1. abb
   - take zero repetitions of $(a^* b)^*$.

2. babb

b + abb = babb.

3. ababb

ab + abb = ababb

4. aababb

aab + abb = aababb

5. bbabb

b + b + abb = bbabb.

answer:

$L = \{(a^*b)^* abb\}$

five valid strings:

abb, babb, ababb, aababb, bbabb

c) $(a^*b)^* aba (a+b)^*$

Step 1: $(a^*b)^*$

can produce zero or more blocks such as:

b
a b
a a b
a a a b

Step 2: middle part is fixed.

aba

Step 3: last part

$(a+b)^*$

means any combination of a & b including the empty string

so, the language has the form:

$(a^*b)^* aba (a+b)^*$

Scanned with OKEN Scanner

five valid strings.

1. aba.

Let, $(a*b)^* = \varepsilon$

and $(a+b)^* = \varepsilon$

So, aba

2. baba.

~~let~~ take b ~~be~~ from the 1st part:

~~ab+~~ b + aba = baba

3. ababa

take ab from the 1st ~~person~~ part:

ab + aba = ababa.

4. aababa

take aab from the 1st part,

aab + aba = aababa.

5. abab.

take aba from the 1st part,

aba + b = abab

answer:

$L = \{(a*b)^* aba (a+b)^*\}$

five valid strings:

aba, baba, ababa, aababa, abab

d) $(aab)^*$

Step 1:

aab

The '*' mean that this block can occur Zero or more times.

ooo, $(aab)^* = \varepsilon, aab, aabaab, aabaabaab, \ldots$

five valid strings

Repetition	String
0 times	ε
1 time	aab
2 times	aabaab
3 times	aabaabaab
4 times	aabaabaab.

Answer, $L = \{(aab)^n \mid n \geq 0\}$

e) $(aba)^* + (a+b)^*$

1st, $(aba)^*$

Produces: ε

aba

abaaba

abaabaaba

. . .

2nd, $(a+b)^*$

ooo, $(aba)^* + (a+b)^* = (a+b)^*$

five valid strings,

$\varepsilon, a, b, ab, ba$

Scanned with OKEN Scanner

answer, $L = (a+b)^*$

Q.3) (i) find the numer of tokens,

given:

```
main()
{ int a = 10;
  Char b = "abc";
  int c = 30;
  Char d = "xyz";
  in /*comment*/ tm = 40.5;
}
```

line 1.

main()

Tokens: count = 3.

lin 2: {

count = 1.

total: 3+1=4.

line 3: int a = 10;

count = 5.

total: 4 + 5 = 9.

line 4: Char b = "abc";

count = 5.

total = 9+5 = 14.

line 5: int c = 30;

count = 6.

total = 20.

line 6:

char d = "xyz";

Count = 6.

total = 20+6 = 26.

line 7:

in /* comment */ tm = 40.5;

Count = 6.

total = 32.

final line: }

Count = 1

∴, 32+1 = 33.

Sol; No, of tokens = 33

ii). Identify the lexical errors.

given:

```
void main()
{ int x = 10, y = 20;
  char *a;
  a = &x;
  x = 1xab;
}
```

Lexical error : 1xab., because it is an invalid numeric literal.

Scanned with OKEN Scanner

Q.4) Find regular expressions,

The alphabet is:

$$\Sigma = \{0, 1\}.$$

(i) Strings having an even no. of 0's

~~Reg~~ RE is: $\boxed{1^*(01^*01^*)^*}$

The two 0's occur together as a pair, with any number of 1's between or around them,

So, the number of 0's is always:

0, 2, 4, 6, ...

Eg:

ε
1
11
00
101
1001
01010.

ii) Strings containing at least one 1.

RE: $\boxed{(0|1)^*1(0|1)^*}$

The middle 1 guarantees that at least one 1 occurs.

Eg:

1
01
10
101
00100
11000.

iii) Strings in which both the number of 0's and number of 1's are odd.

- No. of 0's : odd
- No. of 1's : odd.

$A = (00|11)$

$B = (01|10)$

RE is: $\boxed{A^*BA^*(BA^*BA^*)^*}$

Sub, A & B:

$(00|11)^*(01|10)(00|11)^*((01|10)(00|11)^*(01|10)(00|1$

iv) Strings that start with 1 & end with 0.

RE: $\boxed{1(0|1)^*0}$

- 1 → String must start with 1.
- $(0|1)^*$ → any combination can occur in the middle.
- 0 → String must end with 0.

Eg:

10
100
110
1010
1110
10010.

v) Strings that do not contain consecutive 1's

consecutive 1's,

1 1 .

R.E : $(0|10)^*(\varepsilon|1)$

Q.5). Lexical Analyzer.

Input:

int var123 = 12345; float var456 = 123.45;

if (var123 == var456) { return; }.

a) No. of tokens.

1. int.
2. var123
3. =
4. 12345
5. ;
6. float
7. var456
8. =
9. 123.45.
10. ;
13. var123
14. ==
15. var456
16.)
17. {
18. return
19. ;
20. }.

Scanned with OKEN Scanner

Therefore, 20 tokens

b) Categorize the tokens.

Keywords:
int
float
if
return.
Count = 4

Operators
=
=
=
Count = 3

Separators.
;
,
(
)
{
}
Count = 7

Identifiers.
Var123
Var456
var123
Var456
Count = 4

Integer literal.
12345
Count = 1

floating-point literal
123.45.
Count = 1

Total tokens: 20

Scanned with OKEN Scanner

c) FSM state transitions.

Approximate transitions can be counted according to the characters in each token.

Token type	Transitions
Keywords	16
Identifiers	24
Separators	7
Operators	4.
Integer	5.
float	6.
Total	62.

Approximately 62 state transitions.